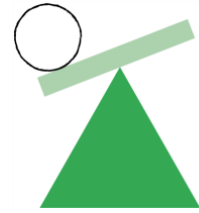


Errors + Grace Failure

Chapter worksheet



Instructions

Block out time to get as many cross-functional leads as possible together in a room to work through these exercises & checklists.

Exercises

1. Error audit [~1 hour]

Collect canonical error examples to define existing and potential errors and solutions.

2. Quality assurance [~30 minutes]

Prioritize how you'll test and monitor errors and reporting so you can hear from your users early and often.

1. Error audit

As a team, brainstorm what kinds of errors users could encounter. If your team has a working prototype of your feature, try to add current examples.

Use the template below to start collecting error examples so your team has a shared understanding about the different error types and solutions your model could produce.

Error	Error Type	User Stakes	Evidence	Users
Missing requests dependency	Background Error	Medium	Screenshot of traceback: <code>ModuleNotFoundError</code>	Developer
Missing <code>SEC_USER_AGENT</code>	System Limitation	High	HTTP 403 message denial from SEC EDGAR API	Developer
Logging <code>KeyError</code> (<code>run_id</code>)	System Limitation	High	Log formatter error screenshot	Developer
No filings found	Context Error	Medium	Warning output that filings not found	Analyst
Missing raw HTML (parse fails)	Background Error	High	<code>FileNotFoundError</code> during parse stage	Developer/Analyst

Error sources

Take each error identified above through these questions to determine the source of the error:



Input error signals

- SEC_USER_AGENT incorrectly assumed to be automatic
- Invalid ticker/year results in unexpected empty responses

Relevance error signals

- Chosen filing range contains no results
- Output not matching user's mental model of completeness

System hierarchy error

- Logging configuration mismatch causes unclear failure origin
- Dependent pipeline stages hide earlier step failures

Failure state

- Parse and index stages become unusable if fetch fails silently
- Tool stops functioning due to unhandled exceptions

Error resolution

Once you have identified the source or sources of the error, complete the sections below for each of the errors in the template with your team's plan for improving / reducing the identified error. Create as many copies as you need to cover all your identified errors.

Error #1 — Missing requests dependency

Why the user thinks this is an error:

The pipeline fails immediately and shows a traceback, so the user assumes the entire tool is broken.

Solution type:

Better documentation + actionable setup feedback

User path (what they should do):

Install dependencies using:

`pip install -r requirements.txt`

Opportunity for improvement:

Add a dependency check during startup and a friendly message if missing.

Error #2 — Missing SEC_USER_AGENT (HTTP 403)

Why the user thinks this is an error:

The user receives a 403 access denied message with no context, so they assume the fetch logic is faulty.

Solution type:

Feedback + User guidance

User path:

Set required variable:

`export SEC_USER_AGENT="yourname@example.com"`

Opportunity for improvement:

Validate environment variables before running and guide the user to fix it.

Error #3 — Logging KeyError: 'run_id'

Why the user thinks this is an error:

The tool prints logging tracebacks, which look like system failure even though core functionality may continue.

Solution type:

System fix (logging configuration)

User path:

Ensure consistent use of the logging adapter and matching logging.yaml configuration.

Opportunity for improvement:

Provide default fields when missing to avoid crashes.

Error #4 — “No filings found” warning

Why the user thinks this is an error:

The user expected filings based on the year/ticker, so empty results feel like a tool issue.

Solution type:

User control (adjust inputs)

User path:

Try another time window or different ticker; verify filings exist for requested period.

Opportunity for improvement:

Suggest valid alternative years/tickers directly in the warning message.

Error #5 — Missing Raw HTML (Parse step fails)

Why the user thinks this is an error:

The parse script crashes, so they assume the parsing logic is broken.

Solution type:

System fix (step-ordering enforcement)

User path:

Run the fetch step first to ensure raw files exist:

python scripts/fetch_filings.py ...

Opportunity for improvement:

Add a pre-check in parsing to block execution if artifacts are missing.

2. Quality assurance

Getting your feature into users' hands is essential for identifying errors that your team, as expert users, may never encounter. Meet as a team to prioritize how you want to monitor errors reported by users so that your model is being tested and criticized by your users early and often.

As you have this discussion, consider all potential sources of error reporting:

- Reports sent to customer service
- Comments and reports sent through social media channels
- In-product metrics
- In-product surveys
- User research (out-of-product surveys, deep dive interviews, diary studies, etc.)

Goal	Method	Review Frequency	Start Date	End/Review
Ensure pipeline runs successfully E2E	Review logs + check output counts	Weekly	Start of development	Ongoing
Reduce dependency/config failures	Pre-run validation testing	Weekly	After MVP build	Continuous
Improve error messaging clarity	Collect recurring logs from users	Monthly	After deployment	Continuous
Avoid empty fetch runs	Detect no-results cases and advise user	Weekly	Next sprint	Until stable